

v1.4 — REST API & OpenAPI Specification

Excellent. v1.4 is where the architecture becomes a concrete contract between the frontend, backend, workers, and future integrations. I'll define the API conventions first and then the endpoint/schema surface that the FastAPI implementation should follow.

ILCMS — Icelandic Legal Case Management System

Technical Specification v1.4 — REST API & OpenAPI Specification

Document version: 1.4

Status: Engineering Specification

Target market: Iceland

API style: REST / JSON

Framework: FastAPI

API specification: OpenAPI 3.x

Primary API version: v1

Related specifications: v1.1, v1.2, v1.3

1. Purpose

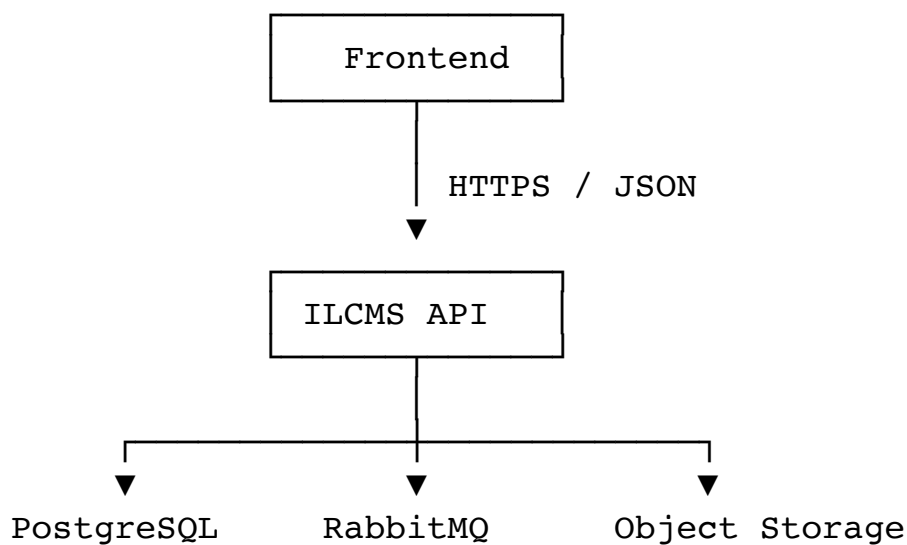
This specification defines the external REST API contract for ILCMS.

The API shall provide controlled access to:

- authentication context
- organizations
- users
- cases
- parties
- classifications
- documents
- document versions
- tasks
- deadlines
- notes

- communications
- notifications
- search
- audit information
- asynchronous jobs
- exports
- system health

The API shall serve as the stable contract between:



The API shall also provide a controlled integration point for future external systems.

2. API Design Principles

The API shall follow these principles:

1. RESTful resource-oriented design.
2. Explicit API versioning.
3. JSON for normal request/response bodies.
4. HTTPS only in non-local environments.
5. Strong authentication.
6. Explicit authorization on every protected resource.
7. Tenant isolation.
8. Consistent error responses.

9. Consistent pagination.
10. Consistent filtering and sorting.
11. Idempotency for retryable operations.
12. No exposure of internal database implementation.
13. No exposure of internal infrastructure details.
14. No sensitive information in error messages.
15. OpenAPI documentation generated from the implementation.
16. Backward-compatible evolution.
17. Icelandic UTF-8 support throughout.

3. Base URL

The production API shall use a deployment-specific hostname.

Logical structure:

`https://api.example.is/api/v1/`

Development:

`http://localhost:8000/api/v1/`

The hostname shall not be hard-coded into the frontend application.

It shall be supplied through environment-specific configuration.

4. API Versioning

The initial API version is:

`/api/v1`

Examples:

`GET /api/v1/cases`

`GET /api/v1/cases/{case_id}`

`POST /api/v1/cases`

Breaking changes require a new major API version.

For example:

`/api/v2`

Minor compatible changes do not require a new API version.

5. Content Type

JSON requests shall use:

Content-Type: application/json

Responses shall normally use:

Content-Type: application/json

File upload/download operations are exceptions.

6. Character Encoding

All JSON shall use UTF-8.

The API shall correctly accept and return Icelandic characters including:

Á É Í Ó Ú Ý Þ Æ Ö Ð

á é í ó ú ý þ æ ö ð

Example:

```
{
  "display_name": "Guðrún Ólafsdóttir"
}
```

7. Authentication

Protected endpoints shall require an authenticated identity.

Preferred mechanism:

OAuth 2.0 / OpenID Connect

Requests shall use:

Authorization: Bearer <access-token>

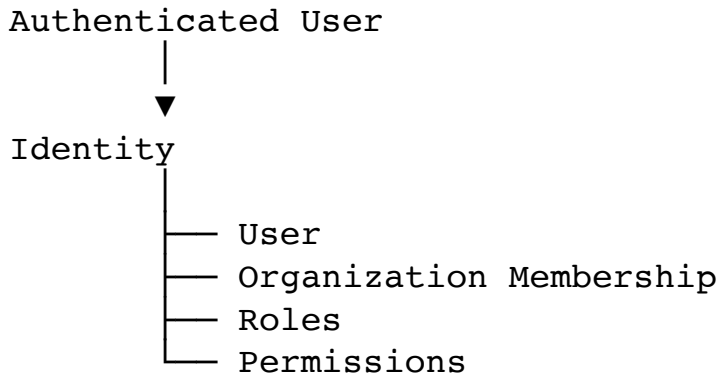
The backend shall validate:

- issuer
- audience
- signature
- expiration
- required claims
- tenant membership

8. Authentication Context

The backend shall construct an authorization context from the authenticated identity.

Conceptually:



Client requests shall not be allowed to override the authenticated user's organization context.

9. Current User

The API shall provide:

GET /api/v1/me

Example response:

```
{
  "id": "uuid",
  "email": "user@example.is",
  "display_name": "Guðrún Ólafsdóttir",
  "locale": "is-IS",
  "timezone": "Atlantic/Reykjavik",
  "organizations": [
    {
      "id": "uuid",
      "name": "Example Legal ehf."
    }
  ]
}
```

Sensitive identity-provider claims shall not be returned unless required.

10. Organization Selection

Where a user belongs to multiple organizations, the active organization shall be explicitly selected.

Possible mechanism:

X-Organization-ID: <uuid>

The backend must verify that the authenticated user has membership in the selected organization.

An organization ID supplied by the client is never sufficient by itself to establish authorization.

11. Authorization

Authorization shall be evaluated at two levels.

Endpoint authorization

Does the user have the required permission?

Object authorization

Does the user have access to this specific resource?

Example:

User

↓

organization membership

↓

permission: case.read

↓

specific case access

12. HTTP Methods

Operation	Method
List	GET
Retrieve	GET
Create	POST
Partial update	PATCH
Replace	PUT where required
Delete	DELETE
Action	POST to action endpoint where appropriate

The API should avoid RPC-style endpoints except where an operation is genuinely an action.

13. Resource Naming

Resources shall use plural nouns.

Examples:

```
/cases  
/parties  
/documents  
/tasks  
/deadlines  
/notifications
```

Path parameters shall use UUIDs:

```
/cases/{case_id}
```

14. Standard Response Envelope

For individual resources, responses should normally return the resource directly.

Example:

```
{  
  "id": "...",  
  "title": "Example case",  
  "status": "OPEN"  
}
```

Collection responses shall use a standard structure.

```
{  
  "items": [],  
  "page": 1,  
  "page_size": 50,  
  "total": 0  
}
```

15. Pagination

Collection endpoints shall support:

```
page  
page_size
```

Example:

```
GET /api/v1/cases?page=1&page_size=50
```

Recommended maximum:

```
page_size <= 100
```

The exact limit shall be configurable.

Cursor pagination may be introduced for very large collections.

16. Pagination Metadata

Example:

```
{
  "items": [],
  "page": 1,
  "page_size": 50,
  "total": 157
}
```

Future cursor-based responses may use:

```
{
  "items": [],
  "next_cursor": "...
}
```

17. Filtering

Filtering shall use explicit query parameters.

Example:

```
GET /api/v1/cases?status=OPEN
```

Multiple filters:

```
GET /api/v1/cases?
  status=OPEN&
  priority=HIGH&
  assigned_to=<uuid>
```

Only supported filter fields shall be accepted.

18. Sorting

Example:

```
GET /api/v1/cases?sort=-updated_at
```

The backend shall maintain a whitelist of sortable fields.

Example:

```
created_at
updated_at
case_number
title
priority
```

Arbitrary SQL expressions shall never be accepted.

19. Search

General search endpoint:

```
GET /api/v1/search?q=<query>
```

Example:

```
GET /api/v1/search?q=Þórðarson
```

Search results shall contain only resources the user is authorized to access.

20. Standard Error Schema

All API errors shall use:

```
{
  "error": {
    "code": "CASE_NOT_FOUND",
    "message": "The requested case could not be found.",
    "request_id": "550e8400-e29b-41d4-a716-446655440000"
  }
}
```

The `code` is machine-readable.

The `message` is intended for controlled client display.

21. Error Codes

Initial error catalog:

```
AUTHENTICATION_REQUIRED
INVALID_TOKEN
TOKEN_EXPIRED
ACCESS_DENIED
RESOURCE_NOT_FOUND
```

VALIDATION_ERROR
CONFLICT
INVALID_STATE
RATE_LIMITED
FILE_TOO_LARGE
UNSUPPORTED_FILE_TYPE
PROCESSING_FAILED
INTERNAL_ERROR
Domain-specific errors may be added.

22. HTTP Error Mapping

Recommended mapping:

Condition	HTTP
Authentication missing	401
Authentication invalid	401
Permission denied	403
Resource inaccessible/not found	404
Invalid input	422
State conflict	409
Duplicate resource	409
Rate limit	429
Unexpected failure	500

For sensitive resources, 404 may be returned instead of 403 where revealing resource existence would itself be sensitive.

23. Request ID

Every API request shall have a request ID.

Example:

X-Request-ID: 4a9e...

The server shall generate one if absent.

The ID shall appear in:

- application logs

- distributed traces
- relevant audit events
- error responses

24. Idempotency

Operations susceptible to retry shall support:

Idempotency-Key: <uuid>

This is particularly important for:

- document creation
- exports
- external integrations
- expensive asynchronous operations

25. Case API

Base endpoint:

`/api/v1/cases`

Supported operations:

GET `/cases`

POST `/cases`

GET `/cases/{case_id}`

PATCH `/cases/{case_id}`

DELETE `/cases/{case_id}`

Deletion shall be subject to authorization and retention rules.

26. Create Case

POST `/api/v1/cases`

Request:

```
{
  "title": "Contract dispute",
  "description": "Dispute concerning contract obligations.",
  "case_type": "CONTRACT",
  "classification_id": "uuid",
```

```
    "priority": "NORMAL"
}
```

Response:

201 Created

```
{
  "id": "uuid",
  "case_number": "2026-00157",
  "title": "Contract dispute",
  "status": "OPEN",
  "priority": "NORMAL"
}
```

The backend generates the case number.

27. Retrieve Case

GET /api/v1/cases/{case_id}

The response should provide the case's principal metadata.

Example:

```
{
  "id": "uuid",
  "case_number": "2026-00157",
  "title": "Contract dispute",
  "description": "...",
  "case_type": "CONTRACT",
  "status": "ACTIVE",
  "priority": "HIGH",
  "opened_at": "2026-09-01T10:00:00Z",
  "responsible_user_id": "uuid",
  "created_at": "...",
  "updated_at": "..."
}
```

28. Update Case

PATCH /api/v1/cases/{case_id}

Only supplied fields are changed.

Example:

```
{
  "title": "Updated contract dispute",
  "priority": "HIGH"
}
```

```
}
```

Every update shall be authorized and audited.

29. Case State Transition

State-changing operations should use explicit action endpoints where the transition itself has business significance.

Example:

```
POST /api/v1/cases/{case_id}/close
```

Response:

```
{
  "id": "uuid",
  "status": "CLOSED",
  "closed_at": "...",
}
```

This makes the business operation explicit and auditable.

30. Case Assignment

```
POST /api/v1/cases/{case_id}/assign
```

Request:

```
{
  "user_id": "uuid"
}
```

The user must belong to the relevant organization and have the required role/permission.

The operation generates an audit event.

31. Case Users

```
GET /api/v1/cases/{case_id}/users
```

```
POST /api/v1/cases/{case_id}/users
```

```
DELETE /api/v1/cases/{case_id}/users/{user_id}
```

Create request:

```
{
  "user_id": "uuid",
  "role": "PARALEGAL"
}
```

32. Case Parties

```
GET    /api/v1/cases/{case_id}/parties
POST   /api/v1/cases/{case_id}/parties
DELETE /api/v1/cases/{case_id}/parties/{party_id}
Create relationship:
```

```
{
  "party_id": "uuid",
  "role": "CLIENT",
  "is_primary": true
}
```

33. Party API

Base endpoint:

```
/api/v1/parties
```

Operations:

```
GET    /parties
POST   /parties
GET    /parties/{party_id}
PATCH /parties/{party_id}
```

34. Create Party

Example:

```
{
  "party_type": "PERSON",
  "display_name": "Jón Þórðarson",
  "given_name": "Jón",
  "family_name": "Þórðarson",
  "full_legal_name": "Jón Þórðarson"
}
```

The backend shall not require `family_name`.

35. Party Search

```
GET /api/v1/parties?q=Jón
```

Search shall support Icelandic characters and shall be authorization-aware.

36. Classifications API

```
GET /api/v1/classifications
POST /api/v1/classifications
GET /api/v1/classifications/{classification_id}
PATCH /api/v1/classifications/{classification_id}
Tree retrieval may be supported:

GET /api/v1/classifications/tree
```

37. Document API

Base endpoint:

```
/api/v1/documents
```

Operations:

```
GET /documents/{document_id}
POST /cases/{case_id}/documents
PATCH /documents/{document_id}
GET /documents/{document_id}/versions
GET /documents/{document_id}/download
Document deletion shall be heavily restricted.
```

38. Document Upload

Recommended workflow:

```
POST /api/v1/cases/{case_id}/documents/upload
```

Request:

```
{
  "title": "Court decision",
  "document_type": "COURT_DECISION",
  "filename": "court_decision.pdf",
  "mime_type": "application/pdf",
  "size": 1048576
}
```

Response:

```
{
  "document_id": "uuid",
  "version_id": "uuid",
  "upload_url": "...",
}
```

```
    "expires_at": "...",
    "status": "UPLOAD_PENDING"
}
```

The URL shall be short-lived.

39. Upload Completion

After successful object-storage upload:

POST /api/v1/documents/{document_id}/upload-complete

The backend shall verify the object exists and initiate processing.

Response:

```
202 Accepted
{
  "document_id": "uuid",
  "status": "PROCESSING",
  "job_id": "uuid"
}
```

40. Document Processing Status

GET /api/v1/documents/{document_id}/processing

Example:

```
{
  "status": "PROCESSING",
  "steps": [
    {
      "type": "VIRUS_SCAN",
      "status": "COMPLETED"
    },
    {
      "type": "OCR",
      "status": "RUNNING"
    },
    {
      "type": "INDEXING",
      "status": "QUEUED"
    }
  ]
}
```

41. Document Download

GET /api/v1/documents/{document_id}/download

The API should return a short-lived signed URL rather than proxying large files through the API server.

Example:

```
{
  "download_url": "...",
  "expires_at": "...",
}
```

The download operation shall generate an audit event where required by policy.

42. Document Versions

GET /api/v1/documents/{document_id}/versions

Response:

```
{
  "items": [
    {
      "id": "uuid",
      "version_number": 3,
      "checksum_sha256": "...",
      "file_size": 123456,
      "created_at": "..."
    }
  ]
}
```

Versions are immutable.

43. Create Document Version

POST /api/v1/documents/{document_id}/versions

The system shall create the next version number.

Clients shall not select arbitrary version numbers.

44. Tasks API

GET /api/v1/tasks

```
POST /api/v1/tasks
GET /api/v1/tasks/{task_id}
PATCH /api/v1/tasks/{task_id}
POST /api/v1/tasks/{task_id}/complete
```

45. Create Task

```
{
  "case_id": "uuid",
  "title": "Review contract",
  "description": "Review clauses 5–8.",
  "assigned_to": "uuid",
  "priority": "HIGH",
  "due_at": "2026-09-15T16:00:00+00:00"
}
```

The backend validates that the assigned user can access the case.

46. Complete Task

```
POST /api/v1/tasks/{task_id}/complete
```

Response:

```
{
  "id": "uuid",
  "status": "COMPLETED",
  "completed_at": "..."
}
```

47. Deadlines API

```
GET /api/v1/deadlines
POST /api/v1/deadlines
GET /api/v1/deadlines/{deadline_id}
PATCH /api/v1/deadlines/{deadline_id}
POST /api/v1/deadlines/{deadline_id}/complete
```

48. Deadline Creation

Example:

```
{
  "case_id": "uuid",
```

```
"title": "Response deadline",
"deadline_type": "COURT",
"due_at": "2026-10-15T16:00:00+00:00",
"source_document_id": "uuid"
}
```

The API shall store the source but shall not infer legal validity.

49. Notes API

```
GET /api/v1/cases/{case_id}/notes
POST /api/v1/cases/{case_id}/notes
GET /api/v1/notes/{note_id}
PATCH /api/v1/notes/{note_id}
```

Note visibility shall be authorization-controlled.

50. Communications API

```
GET /api/v1/cases/{case_id}/communications
POST /api/v1/cases/{case_id}/communications
GET /api/v1/communications/{communication_id}
```

Communication records should reference associated documents where appropriate.

51. Notification API

```
GET /api/v1/notifications
POST /api/v1/notifications/{notification_id}/read
POST /api/v1/notifications/read-all
```

Notifications are user-specific.

A user must never be able to retrieve another user's notifications.

52. Search API

General search:

```
GET /api/v1/search?q=<query>
```

Optional:

```
type=case
type=party
type=document
```

Example:

```
GET /api/v1/search?q=Þjóðskjalasafn&type=document
```

Results:

```
{
  "items": [
    {
      "type": "document",
      "id": "uuid",
      "title": "Archive correspondence",
      "case_id": "uuid",
      "highlight": "..."
    }
  ]
}
```

53. Search Security

Search shall not be implemented as:

```
search all records
```

↓

```
filter results afterwards
```

Instead, authorization must form part of the search query or be enforced by a secure search-access layer.

This is essential to prevent information leakage.

54. Audit API

Audit access shall be restricted.

```
GET /api/v1/audit-events
```

```
GET /api/v1/cases/{case_id}/audit-events
```

Filters may include:

```
event_type
```

```
actor_user_id
```

```
resource_type
```

```
resource_id
```

```
from
```

```
to
```

55. Audit Response

Example:

```
{
  "items": [
    {
      "id": "uuid",
      "event_type": "CASE_UPDATED",
      "resource_type": "case",
      "resource_id": "uuid",
      "actor_user_id": "uuid",
      "timestamp": "2026-09-02T08:00:00Z",
      "metadata": {
        "field": "priority"
      }
    }
  ]
}
```

Sensitive metadata shall be filtered according to authorization.

56. Background Jobs API

Users may need to inspect asynchronous operations.

GET /api/v1/jobs/{job_id}

Response:

```
{
  "id": "uuid",
  "type": "DOCUMENT_PROCESSING",
  "status": "RUNNING",
  "created_at": "...",
  "started_at": "..."
}
```

Internal infrastructure details shall not be exposed.

57. Export API

Exports shall be asynchronous.

POST /api/v1/exports

GET /api/v1/exports

GET /api/v1/exports/{export_id}

GET /api/v1/exports/{export_id}/download

Create request:

```
{
  "type": "CASE_PACKAGE",
  "case_id": "uuid"
}
```

Response:

202 Accepted

```
{
  "export_id": "uuid",
  "status": "QUEUED"
}
```

58. Export Security

Exports shall require:

- explicit permission
- authorization for the requested resources
- audit logging
- controlled storage
- expiration
- secure download

Export scope shall be validated server-side.

59. Organization API

GET /api/v1/organizations

GET /api/v1/organizations/{organization_id}

PATCH /api/v1/organizations/{organization_id}

Administrative operations shall require appropriate permissions.

60. User Administration API

GET /api/v1/users

GET /api/v1/users/{user_id}

PATCH /api/v1/users/{user_id}

Potential administrative endpoints:

POST /api/v1/users/{user_id}/activate

POST /api/v1/users/{user_id}/deactivate
POST /api/v1/users/{user_id}/roles
These operations shall be strongly audited.

61. Role and Permission API

Administrative users may access:

GET /api/v1/roles
GET /api/v1/permissions
POST /api/v1/roles
PATCH /api/v1/roles/{role_id}
System-defined roles shall not be freely modified.

62. Health API

Liveness:

GET /live

Readiness:

GET /ready

These endpoints should normally not require authentication but must not expose sensitive information.

Example:

```
{  
  "status": "ok"  
}
```

63. API Authentication vs Health

Health endpoints shall not return:

- database credentials
- hostnames
- connection strings
- environment secrets
- internal IP addresses
- dependency stack traces

64. API Rate Limiting

Rate limiting should be applied especially to:

authentication
search
exports
document processing
AI operations
administrative operations
A rate-limit response:

429 Too Many Requests
may include:

Retry-After: 30

65. File Upload Security

The API shall enforce:

- maximum file size
- permitted file types
- object-storage authorization
- malware scanning
- checksum generation
- filename sanitization

The original filename shall be treated as untrusted user input.

66. Filename Handling

A filename such as:

`../../../../etc/passwd`
must never become a filesystem path.

Object-storage keys shall be generated by the backend.

Recommended:

`organizations/{org_id}/documents/{document_id}/{version_id}`

rather than using the user filename as the storage key.

67. API and Object Storage

The API should never expose permanent storage credentials.

Clients receive:

short-lived signed URL
rather than:

S3 credentials

68. Conditional Requests

Future API versions may support:

ETag

If-Match

If-None-Match

This is particularly useful for optimistic concurrency.

Example:

If-Match: "version-7"

An outdated client would receive:

409 Conflict

69. Optimistic Concurrency API

Where supported, resources may expose:

```
{  
  "id": "uuid",  
  "version": 7  
}
```

Update:

```
{  
  "title": "New title",  
  "version": 7  
}
```

The backend rejects the update if the current version differs.

70. Date and Time Format

API timestamps shall use RFC 3339/ISO 8601 format.

Example:

```
2026-09-02T08:30:00Z
```

or:

```
2026-09-02T08:30:00+00:00
```

The API shall avoid naive timestamps.

71. Date-Only Values

Dates without time shall use:

```
YYYY-MM-DD
```

Example:

```
{  
  "document_date": "2026-09-02"  
}
```

The backend shall distinguish date-only values from timestamps.

72. Enum Representation

Enumerations shall use stable machine-readable values.

Example:

```
{  
  "status": "IN_PROGRESS"  
}
```

The frontend may translate this into Icelandic:

```
Í vinnslu
```

This avoids coupling API values to presentation language.

73. Localization

User-visible API-generated messages should support localization.

The preferred architecture is:

API

↓

stable error code

↓

Frontend localization

↓

Icelandic/English display

For example:

CASE_NOT_FOUND

rather than relying exclusively on:

Málið fannst ekki

as the machine-readable response.

74. API Documentation

FastAPI shall generate OpenAPI documentation.

Development environments may expose:

/docs

/redoc

/openapi.json

Production exposure shall be configurable.

Sensitive internal API documentation should not automatically be publicly accessible.

75. OpenAPI Tags

The API should be grouped into tags:

Authentication

Organizations

Users

Cases

Parties

Classifications

Documents

Tasks

Deadlines

Notes

Communications

Notifications

Search

Audit
Jobs
Exports
Health

76. OpenAPI Security Scheme

The generated OpenAPI document should define:

```
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

Protected endpoints shall reference the security scheme.

77. Example OpenAPI Structure

Conceptually:

```
openapi: 3.1.0
```

```
info:
  title: ILCMS API
  version: 1.0.0
```

```
servers:
  - url: https://api.example.is/api/v1
```

```
paths:
  /cases:
    get:
      tags:
        - Cases
    post:
      tags:
        - Cases
```

The actual specification shall be generated and validated automatically.

78. Schema Naming

Pydantic schemas shall distinguish operations where necessary.

For example:

CaseCreate

CaseUpdate

CaseResponse

CaseListItem

Avoid using one giant model for every operation.

79. Pydantic Example

Conceptual model:

```
class CaseCreate(BaseModel):  
    title: str  
    description: str | None = None  
    case_type: str | None = None  
    classification_id: UUID | None = None  
    priority: CasePriority = CasePriority.NORMAL
```

The database model shall not be exposed directly through the API.

80. API Schema Separation

The following layers shall remain separate:

HTTP Request

↓

Pydantic Schema

↓

Domain/Application Model

↓

SQLAlchemy Model

This prevents accidental exposure of internal database fields.

81. API Field Exposure

Internal fields should not automatically become API fields.

For example, the database may contain:

internal_processing_error

storage_bucket

internal_job_id

These should not necessarily appear in public API responses.

82. Security-Sensitive Fields

The API shall never return:

- password hashes
- access tokens
- refresh tokens
- client secrets
- storage credentials
- internal encryption keys
- Vault credentials

unless a specific, controlled protocol explicitly requires such information.

83. API Transaction Semantics

A successful mutation shall return only after the required transactional operation has been committed.

Example:

```
POST case
↓
Validate
↓
Authorize
↓
DB transaction
├─ create case
├─ audit event
└─ outbox event
↓
COMMIT
↓
201 Created
```

84. Asynchronous Operations

Expensive operations shall return:

202 Accepted
rather than keeping HTTP requests open indefinitely.

Examples:

document processing
exports
bulk imports
AI processing
large reports

85. API Timeout

Normal synchronous API operations shall have bounded execution time.

Long-running work shall be converted to asynchronous jobs.

This prevents:

HTTP request
↓
10-minute OCR process
and instead uses:

HTTP request
↓
create job
↓
202 Accepted
↓
background processing

86. Bulk Operations

Bulk endpoints shall be introduced only where they provide significant operational value.

Example:

POST /api/v1/documents/bulk-process
Bulk operations shall be asynchronous.

Every item shall be independently authorized.

87. API Audit Requirements

The following API operations shall normally produce audit events:

Case creation/update/closure
Case assignment
Party creation/update
Document upload
Document download
Document version creation
Task completion
Deadline changes
User/role changes
Exports
Administrative configuration changes
Read auditing may be configurable for particularly sensitive resources.

88. API Security Logging

Security-relevant events shall be logged independently of ordinary request logs where necessary.

Examples:

Invalid token
Authorization failure
Repeated authentication failures
Suspicious cross-tenant access
Unexpected permission escalation

89. CORS

CORS shall allow only explicitly configured frontend origins.

Development may allow:

`http://localhost:3000`

Production should allow only the deployed ILCMS frontend origin.

Wildcard production CORS should not be used.

90. CSRF

If authentication uses browser cookies, CSRF protection shall be enabled.

If the frontend uses bearer tokens exclusively through an appropriate architecture, CSRF considerations differ.

The final authentication implementation shall explicitly document the selected model.

91. API Gateway / Ingress

The Kubernetes ingress layer may provide:

- TLS termination
- request-size limits
- rate limiting
- WAF functionality
- access logging
- security headers

The application must still enforce authorization.

Ingress security is not a substitute for backend security.

92. Backward Compatibility

API changes should follow:

Additive

↓

Deprecate

↓

Migration period

↓

Remove in future major version

Deprecated fields/endpoints should be documented.

93. API Deprecation

Deprecated endpoints may return:

`Deprecation: true`
and appropriate documentation.

Removal shall not occur without a defined migration path.

94. API Testing

Every endpoint shall have automated tests covering:

Happy path
Validation failure
Authentication failure
Authorization failure
Cross-tenant access
Not found
Conflict
Concurrency
Audit generation

95. Contract Testing

The OpenAPI specification shall be treated as an API contract.

CI should validate:

FastAPI implementation
↕
OpenAPI schema
↕
Frontend generated/client models
Breaking API changes should be detected automatically.

96. Frontend Client Generation

The API shall be designed so that a typed frontend client can be generated from OpenAPI.

Possible frontend artifacts:

Case
CaseCreate
CaseUpdate
Party
Document
Task
Deadline

The frontend should not manually duplicate API schemas where generated clients are practical.

97. API Performance

The API shall avoid:

- N+1 database queries
- returning huge collections
- synchronous document processing
- unrestricted search
- unnecessary database joins

Endpoints should return only the fields required by the use case.

98. Response Size

Large resources should be paginated or streamed.

The API should not return:

`10000 cases`
in one request.

Similarly, document binary data should normally not be returned through normal JSON APIs.

99. API Security Boundary

The backend is the authoritative security boundary.

The frontend shall be considered untrusted.

Therefore:

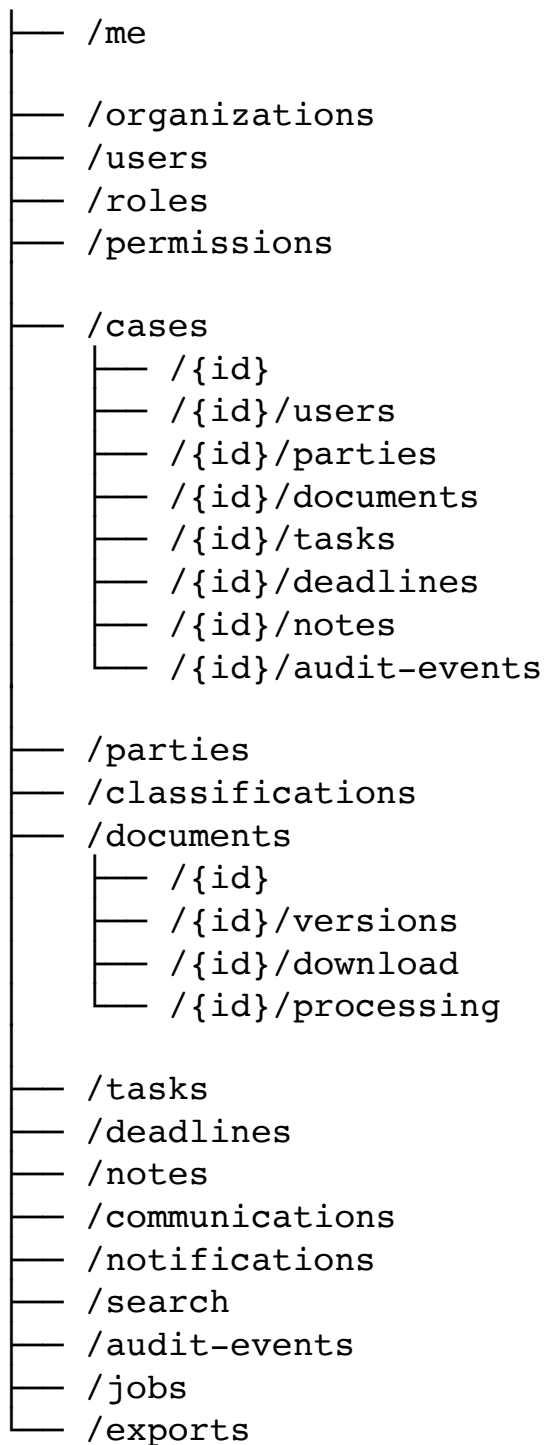
`Frontend hides button`
is not an authorization mechanism.

The backend must independently enforce:

`Authentication`
`Authorization`
`Tenant isolation`
`Object access`
`Validation`
`Audit`

100. Reference API Map

`/api/v1`
|



101. API Implementation Priority

Implementation should proceed in this order:

Phase 1

/me
/cases
/parties
/classifications

Phase 2

/cases/{id}/users
/cases/{id}/parties
/tasks
/deadlines

Phase 3

/documents
/document versions
/upload
/download
/processing

Phase 4

/search
/audit-events
/notifications
/jobs

Phase 5

/exports
/integrations
/advanced administration

102. MVP API

The MVP shall expose at minimum:

Authentication context
Organizations
Users
Cases
Parties
Classifications
Case assignments
Documents
Document versions
Tasks
Deadlines
Search
Audit events
Background jobs

Health endpoints

103. API Definition of Done

An API feature is complete when:

- endpoint implemented
- authentication implemented
- authorization implemented
- tenant isolation tested
- request schema defined
- response schema defined
- error behavior defined
- audit behavior defined
- OpenAPI documentation generated
- unit tests implemented
- integration/API tests implemented
- security tests implemented
- Icelandic UTF-8 tested
- logging implemented
- observability implemented where appropriate
- frontend contract verified

104. OpenAPI Validation Pipeline

CI shall perform:

Generate OpenAPI

↓

Validate OpenAPI

↓

Detect breaking changes

↓

Generate client



Run API contract tests

The generated OpenAPI specification should be treated as a build artifact.

105. API Governance

Changes to public API contracts shall be reviewed.

The project should maintain:

API changelog

API version history

Deprecated endpoint list

Error-code catalog

OpenAPI specification

Authentication documentation

Integration documentation

106. Integration API Future Direction

Future external integrations may use dedicated namespaces:

`/api/v1/integrations/...`

or separate service-to-service APIs.

External integrations shall not receive unrestricted access to the internal API.

Each integration should receive:

- dedicated credentials
- explicit scopes
- explicit organization access
- rate limits
- audit logging
- revocation capability

107. Service-to-Service Authentication

Future service-to-service communication may use:

OAuth 2.0 Client Credentials

or an equivalent workload identity mechanism.

Long-lived static credentials should be avoided.

108. API and Air-Gapped Deployment

The API architecture shall remain compatible with an air-gapped deployment.

The core API must not require external internet access for:

- authentication, where an internal identity provider is deployed
- database access
- document storage
- search
- OCR
- background jobs
- AI functionality

External services shall be optional adapters.

109. API and AI

AI endpoints shall be isolated from ordinary CRUD endpoints.

Potential future namespace:

`/api/v1/ai/...`

Example:

`POST /api/v1/ai/documents/{document_id}/summarize`

Such operations shall:

- require explicit permission
- execute asynchronously
- be audited
- identify the model/version used
- respect document authorization
- avoid sending data to external providers unless explicitly configured

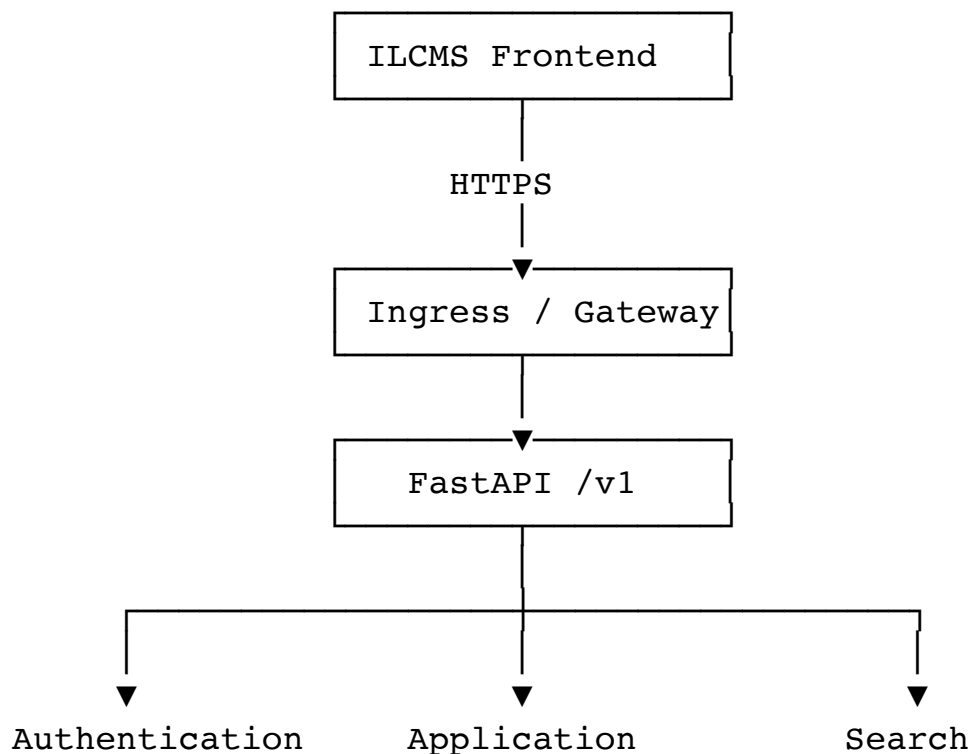
110. API Security Baseline

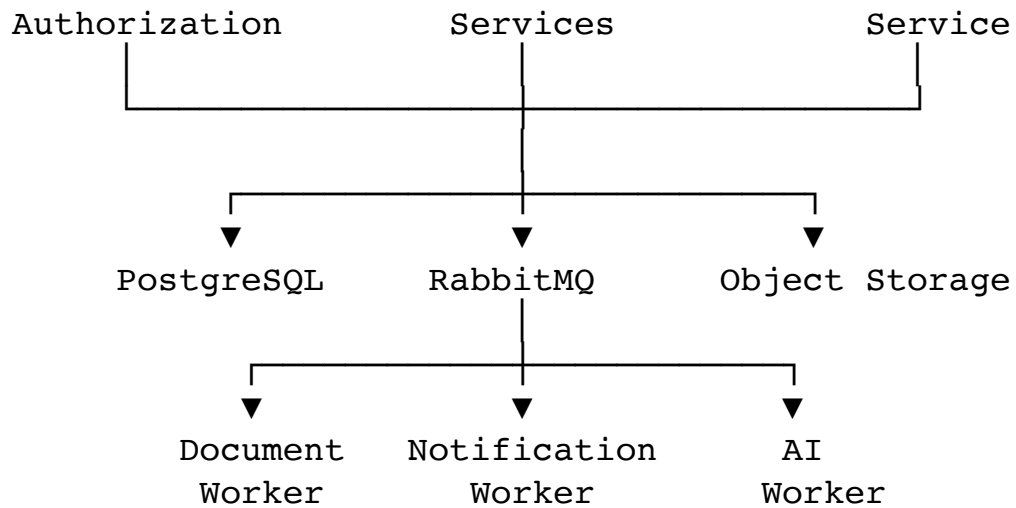
Before production:

- [] HTTPS enforced
- [] Authentication verified
- [] Authorization verified
- [] Tenant isolation tested
- [] BOLA/IDOR tested
- [] Input validation tested
- [] Rate limits configured
- [] CORS restricted
- [] File upload security tested
- [] Error leakage tested
- [] Security headers configured
- [] Audit events verified
- [] Sensitive logging reviewed
- [] OpenAPI validated
- [] Dependency scanning passed
- [] API penetration testing completed

111. Final API Architecture

The completed API architecture should look conceptually like:





112. Version 1.4 Acceptance Criteria

Technical Specification v1.4 is considered implemented when:

1. `/api/v1` is established.
2. Authentication is enforced.
3. Authorization is enforced at endpoint and object level.
4. Tenant isolation is enforced.
5. Standard error responses are implemented.
6. Pagination and filtering conventions are implemented.
7. Case APIs are implemented.
8. Party APIs are implemented.
9. Classification APIs are implemented.
10. Document APIs are implemented.
11. Document upload/download workflows are implemented.
12. Task and deadline APIs are implemented.
13. Search API is implemented.
14. Audit API is implemented.
15. Asynchronous job APIs are implemented.
16. OpenAPI documentation is generated.
17. API contract tests are implemented.

18. Security tests cover cross-tenant and object-level authorization.
19. Icelandic UTF-8 support is tested.
20. The API can operate against the backend/database architecture defined in v1.2/v1.3.

113. Next Specification

The recommended next document is:

Technical Specification v1.5 — Authentication, Authorization & Security Specification

v1.5 should go significantly deeper into the security architecture introduced here.

It should define:

- OIDC architecture
- identity-provider integration
- JWT validation
- token lifecycle
- user provisioning
- organization membership
- RBAC
- permission model
- object-level authorization
- tenant isolation
- PostgreSQL Row-Level Security
- service accounts
- API keys where required
- MFA
- session management
- password policy if local authentication is required
- Vault integration

- secret rotation
- encryption
- audit/security events
- security logging
- rate limiting
- brute-force protection
- privileged/break-glass access
- administrator security
- incident response hooks
- penetration-testing requirements
- threat model

This will establish the **security architecture before implementation of the identity and authorization components**, which is preferable for a legal case-management system handling confidential information.

v1.4 is therefore the API contract layer: v1.3 defines what is stored, v1.4 defines how the rest of the system interacts with it, and v1.5 can now define exactly **who is allowed to do what**. That sequence should make the eventual FastAPI implementation substantially less ambiguous.